

CO - 4 ASSESSMENT TOOL -2

Name : Lanka Kavya
Registration Number : 192472298

Subject Code : CSA0619
Subject Name : DAA

Q17 : A hospital must schedule surgeries and allocate operation theaters efficiently while minimizing patient waiting time. Analyze how scheduling algorithms can optimize resource allocation. Identify constraints such as emergency cases and limited staff. Design a solution using greedy techniques. Discuss real-time scheduling feasibility. Evaluate performance metrics such as utilization and delay reduction. Interpret results in a healthcare management context.

Sol :

Source code :

```
def schedule_surgeries(surgeries, theaters):  
  
    surgeries.sort(key=lambda x: (-x[3], x[2]))  
  
    schedule = []  
  
    available = [0] * theaters  
  
    total_wait = 0  
  
    for s in surgeries:  
  
        patient, duration, arrival, emergency = s  
  
        t = available.index(min(available))  
  
        start = max(arrival, available[t])  
  
        wait = start - arrival  
  
        schedule.append((patient, t + 1, start, start + duration))  
  
        total_wait += wait  
  
        available[t] = start + duration  
  
    avg_wait = total_wait / len(surgeries)  
  
    return schedule, avg_wait
```

```

surgeries = [

    ("P1", 3, 0, 0),

    ("P2", 2, 1, 1),

    ("P3", 4, 2, 0),

    ("P4", 1, 3, 1),

    ("P5", 2, 4, 0)

]

schedule, avg_wait = schedule_surgeries(surgeries, 2)

for s in schedule:

    print(s)

print("Average Waiting Time:", avg_wait)

```

Aim:

To schedule surgeries efficiently and reduce patient waiting time using a greedy algorithm.

Input:

Patient ID, surgery duration, arrival time, emergency status, and number of operation theaters.

Output:

Assigned operation theater, surgery start time, and average waiting time.

Constraints:

- One surgery per theater at a time.
- Emergency surgeries get higher priority.
- Surgery starts only after patient arrival.
- Limited theaters and medical staff.

Algorithm:

1. Give priority to emergency surgeries.
2. Select the earliest available theater.
3. Start surgery as soon as possible.
4. Calculate waiting time.
5. Repeat for all patients.

Greedy Choice:

Choose the highest-priority surgery and the earliest available theater.

Complexity:

Sorting takes $O(n \log n)$. Theater selection takes $O(nT)$, where T is the number of theaters.

Result:

The greedy approach reduces patient waiting time and improves operation theater utilization. It can also be updated in real time when emergency cases or resource changes occur.

Output Screenshot :

```
def schedule_surgeries(surgeries, theaters):
    # Emergency cases first, then shortest waiting time
    surgeries.sort(key=lambda x: (-x[3], x[2]))

    schedule = []
    available = [0] * theaters
    total_wait = 0

    for s in surgeries:
        patient, duration, arrival, emergency = s
        # Select theater available earliest
        t = available.index(min(available))

        start = max(arrival, available[t])
        wait = start - arrival

        schedule.append((patient, t + 1, start))
        total_wait += wait

        available[t] = start + duration

    avg_wait = total_wait / len(surgeries)

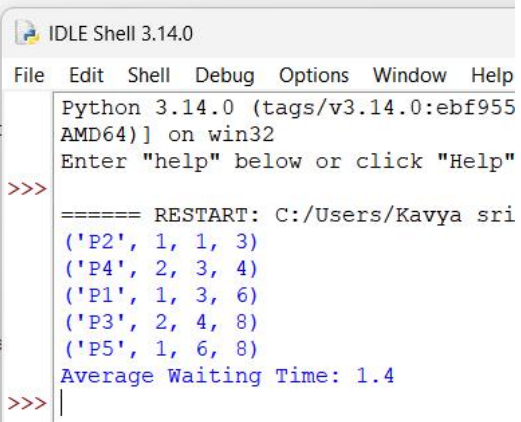
    return schedule, avg_wait

# Patient, Surgery Time, Arrival Time, Emergency
surgeries = [
    ("P1", 3, 0, 0),
    ("P2", 2, 1, 1),
    ("P3", 4, 2, 0),
    ("P4", 1, 3, 1),
    ("P5", 2, 4, 0)
]

schedule, avg_wait = schedule_surgeries(surgeries, 5)

for s in schedule:
    print(s)

print("Average Waiting Time:", avg_wait)
```



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955
AMD64) on win32
Enter "help" below or click "Help"

===== RESTART: C:/Users/Kavya sri
('P2', 1, 1, 3)
('P4', 2, 3, 4)
('P1', 1, 3, 6)
('P3', 2, 4, 8)
('P5', 1, 6, 8)
Average Waiting Time: 1.4
>>> |
```